Práctica 08: integración continua con GitHub Actions

Proyecto: BiblioTech

Fecha: 22/10/2026

Funcionalidad elegida: Límite de préstamos

Repositorio configurado: jorgecahui/BiblioTech

1. Objetivo

Desarrollar y comprobar la regla que impide superar tres préstamos activos por estudiante, incorporar pruebas automáticas y preparar un flujo de integración continua que compile el backend y ejecute esas pruebas ante cambios de código. La práctica incluye un pull request hacia `develop` y una demostración de detección de error y recuperación mediante GitHub Actions.

La implementación local está verificada: **27 pruebas del backend y 24 de la interfaz aprobadas**, además de compilación y empaquetado correctos. Las ramas están publicadas en GitHub y el pull request #1 hacia `develop` está abierto para revisión, con CI en **Success**. También se completó la demostración remota: un error deliberado produjo **Failure** y su corrección produjo **Success**, con reportes que acreditan la detección de la frontera del límite.

2. Regla de negocio y criterios de aceptación

Un estudiante puede mantener **como máximo tres préstamos activos**. Si actualmente tiene cero, uno o dos, puede solicitar un préstamo adicional siempre que esté activo y exista al menos un ejemplar disponible. Si tiene tres o más, el sistema debe rechazar la solicitud antes de modificar los préstamos o el inventario.

Préstamos activos actuales	¿Puede solicitar otro?	Resultado esperado
0	Sí	Se registra el primero si cumple las demás condiciones
1	Sí	Se registra el segundo si cumple las demás condiciones
2	Sí	Se registra el tercero y se descuenta un ejemplar
3	No	Se rechaza el cuarto y se conserva el inventario
Más de 3	No	Se rechazan nuevos préstamos aunque existan datos previos fuera de la regla

El estado del préstamo se cuenta sin distinguir mayúsculas y minúsculas. Los estados `ACTIVO` y `activo` deben participar en el mismo límite. Un estudiante suspendido, un libro agotado o una referencia inexistente no autorizan el registro aunque todavía exista cupo.

3. Implementación

Servicio y validación

En PrestamoService.java se declara `MAX_PRESTAMOS_ACTIVOS = 3`. La misma constante se usa al consultar la disponibilidad y al registrar el préstamo para mantener una única definición del límite.

`puedePrestar` comprueba el estado del estudiante, cuenta sus préstamos activos y devuelve `false` cuando el contador es mayor o igual que tres. Después verifica la disponibilidad del libro. Es una consulta; su resultado representa el estado observado en ese momento y no reserva cupo.

`registrarPrestamo` vuelve a verificar las condiciones dentro de una transacción. Cuando se permite el registro, crea un préstamo con estado `ACTIVO`, asigna la fecha actual y reduce en uno los ejemplares disponibles.

Transacción y bloqueo de filas

El registro tiene `@Transactional`. Las consultas `findByIdForUpdate` de EstudianteRepository.java y LibroRepository.java usan `PESSIMISTIC_WRITE`.

El servicio bloquea primero el estudiante antes de contar sus préstamos. De esta manera, dos registros concurrentes realizados mediante este servicio para el mismo estudiante deben esperar su turno y volver a comprobar el cupo. Posteriormente bloquea el libro para comprobar el stock y descontar el ejemplar. La actualización del inventario y la creación del préstamo forman parte de la misma transacción: un error provoca su reversión.

Estas garantías dependen de utilizar el método transaccional y de que la base de datos aplique los bloqueos. Las pruebas con repositorios simulados comprueban que el servicio llama a las consultas de bloqueo; la comprobación con una base de datos corresponde a las pruebas de integración. El modo de compatibilidad PostgreSQL de H2 no sustituye una validación específica del motor PostgreSQL en un entorno de despliegue.

Conteo de préstamos activos

PrestamoRepository.java declara `countByEstudianteIdAndEstadoIgnoreCase`. Así el conteo de los préstamos del estudiante no omite registros por una diferencia de mayúsculas y minúsculas en el estado.

Rechazo de la operación y respuesta HTTP

LimitePrestamosException.java representa el rechazo por haber alcanzado el máximo. Contiene el número de préstamos activos y el límite permitido, y explica que se debe devolver un libro antes de solicitar otro.

PrestamoExceptionHandler.java convierte esa excepción en una respuesta **HTTP 409 Conflict** mediante `ProblemDetail`. Incluye el título `Límite de préstamos alcanzado` y los campos `prestamosActivos` y `limite`.

En PrestamoController.java, la consulta `GET /api/prestamos/validar` devuelve un booleano. El intento de registro `POST /api/prestamos` es el que genera el conflicto HTTP cuando se alcanza el límite. La protección del registro no depende de haber ejecutado previamente la consulta de validación.

Interfaz Angular y ejecución local

El formulario muestra la regla de máximo tres préstamos activos. Cuando la API rechaza un registro, presenta el campo `detail` de la respuesta HTTP 409. Valida IDs enteros positivos y bloquea botones y campos durante la solicitud para impedir envíos repetidos. Las respuestas asíncronas actualizan la vista de Angular.

PrestamoController.java permite el origen local `http://localhost:4200` mediante CORS para que el navegador reciba la respuesta del backend. La configuración de PostgreSQL admite las variables `DB_URL`, `DB_USER` y `DB_PASSWORD`; la contraseña local no se incorpora al repositorio. Las instrucciones de ejecución están en README.md.

4. Estrategia de pruebas

Las pruebas unitarias se encuentran en PrestamoServiceTest.java. Usan JUnit Jupiter y Mockito para controlar los datos que devuelven los repositorios y comprobar tanto la respuesta como la ausencia de guardados cuando la solicitud se rechaza.

El perfil de pruebas está definido en application-test.properties. Usa una base H2 en memoria, el modo de compatibilidad PostgreSQL y creación/eliminación del esquema. H2 se incorpora con alcance `test` en pom.xml. La prueba BibliotechV2ApplicationTests.java activa ese perfil para cargar el contexto sin depender de una instancia externa de PostgreSQL.

PrestamoIntegrationTest.java usa repositorios y transacciones reales sobre H2. Prepara casos del conteo sin distinguir mayúsculas, el tercer préstamo permitido y el cuarto rechazado, la liberación del cupo al devolver, la reversión de una transacción fallida y dos solicitudes simultáneas que disputan el último cupo. PrestamoControllerTest.java usa MockMvc con un servicio simulado para comprobar el contrato HTTP de validación, registro y conflicto 409.

Matriz de casos

La siguiente tabla describe la cobertura comprobada. Las cifras y los registros de ejecución se encuentran en la sección de verificación final.

Caso	Entrada o condición	Resultado esperado	Comprobación preparada
Cupo disponible	0, 1 y 2 préstamos activos	`puedePrestar` devuelve `true`	Prueba parametrizada con tres valores
Límite exacto	3 préstamos activos	`puedePrestar` devuelve `false`	Prueba de frontera que detecta el cambio >= por >
Datos por encima del máximo	4 y 8 préstamos activos	Se rechaza la validación	Prueba parametrizada de valores superiores
Tercer préstamo permitido	2 préstamos activos y 5 ejemplares	Se registra el tercero; quedan 4 ejemplares	Captura del préstamo guardado y verificación del libro
Cuarto préstamo rechazado	3, 4 y 8 préstamos activos	`LimitePrestamosException`; sin guardar préstamo ni acceder al libro	Prueba parametrizada de registro inválido
Estudiante suspendido	Estado `SUSPENDIDO`	Se rechazan validación y registro	Pruebas que comprueban ausencia de interacción con los demás repositorios
Libro agotado	0 ejemplares disponibles	Se rechazan validación y registro	Comprobación de que el inventario sigue en cero y no se guarda
Estudiante inexistente	Búsqueda vacía	Se informa el error sin registrar	Pruebas de validación y registro
Libro inexistente	Búsqueda vacía y estudiante con cupo	Se informa el error sin registrar	Pruebas de validación y registro
Conteo por estudiante y estado	Préstamos `ACTIVO`, `activo`, `DEVUELTO` y de otro estudiante	Solo se cuentan los dos activos del estudiante elegido	Integración con H2
Devolución	Tres activos; uno pasa a `DEVUELTO`	Se libera un cupo y se permite un nuevo registro	Integración con H2
Reversión de transacción	Error después de registrar y antes del commit	No queda préstamo nuevo y el inventario se conserva	Integración con `TransactionTemplate`
Concurrencia sobre el último cupo	Dos activos y dos solicitudes simultáneas del mismo estudiante	Solo una obtiene el tercer cupo; el total queda en tres	Integración con dos hilos y consultas con bloqueo
Contrato de validación HTTP	El servicio rechaza la validación	HTTP 200 con cuerpo `false`	MockMvc
Contrato de rechazo HTTP	El servicio lanza `LimitePrestamosException`	HTTP 409 y detalle del límite	MockMvc con `PrestamoExceptionHandler`
Contrato de registro HTTP	El servicio registra un préstamo autorizado	HTTP 200 con el préstamo	MockMvc
Carga del contexto	Perfil `test`	Spring inicia con H2	Prueba de contexto

La simulación de repositorios se acompaña de pruebas de integración con H2 para verificar persistencia, transacciones y concurrencia.

Las **24 pruebas de Angular** cubren aplicación, servicio HTTP y formulario: validación de IDs, campos vacíos, mensajes de éxito y rechazo por límite, formato de peticiones, bloqueo durante la espera y prevención de

solicitudes duplicadas. La configuración de Vitest utiliza un hilo de trabajo para evitar el tiempo de espera de arranque de procesos observado en Windows.

Ejemplo real: entrada válida

Este método, copiado de `PrestamoServiceTest`, prueba tres entradas válidas. Los identificadores, los repositorios simulados y los métodos auxiliares pertenecen a la misma clase de pruebas.

```
@ParameterizedTest(name = "Permite validar con {0} préstamos activos")
@ValueSource(longs = {0, 1, 2})
void permiteValidarMientrasNoAlcanzaElLimite(long activos) {
    when(estudianteRepository.findById(ESTUDIANTE_ID))
            .thenReturn(Optional.of(estudianteActivo()));
    when(prestamoRepository.countByEstudianteIdAndEstadoIgnoreCase(ESTUDIANTE_ID, "ACTIVO"))
            .thenReturn(activos);
    when(libroRepository.findById(LIBRO_ID))
            .thenReturn(Optional.of(libroDisponible()));

    assertTrue(service.puedePrestar(ESTUDIANTE_ID, LIBRO_ID));

    verify(prestamoRepository).countByEstudianteIdAndEstadoIgnoreCase(ESTUDIANTE_ID, "ACTIVO");
    verify(prestamoRepository, never()).save(any(Prestamo.class));
}
```

Ejemplo real: entrada inválida

Este segundo método comprueba el límite exacto y valores superiores. También verifica que, al agotarse el cupo, no se consulta el libro ni se guarda un nuevo préstamo.

```
@ParameterizedTest(name = "Rechaza validar con {0} préstamos activos")
@ValueSource(longs = {3, 4, 8})
void rechazaValidarDesdeElTercerPrestamoActivo(long activos) {
    when(estudianteRepository.findById(ESTUDIANTE_ID))
            .thenReturn(Optional.of(estudianteActivo()));
    when(prestamoRepository.countByEstudianteIdAndEstadoIgnoreCase(ESTUDIANTE_ID, "ACTIVO"))
            .thenReturn(activos);
    // El libro válido hace que una regresión del límite falle en assertFalse.
    // Es lenient porque el comportamiento correcto no consulta el libro.
    lenient().when(libroRepository.findById(LIBRO_ID))
            .thenReturn(Optional.of(libroDisponible()));

    assertFalse(service.puedePrestar(ESTUDIANTE_ID, LIBRO_ID));

    verifyNoInteractions(libroRepository);
    verify(prestamoRepository, never()).save(any(Prestamo.class));
}
```

5. Flujo de integración continua

El archivo ci.yml define el flujo `CI - BiblioTech`. Se ejecuta ante un push a `main`, `develop`, `feature/**` o `practica/**`, ante un pull request hacia `develop` y por activación manual cuando GitHub disponga del flujo en la rama predeterminada.

El trabajo `Backend - Java 21 y pruebas` obtiene el código, prepara Java 21 Temurin, utiliza la caché de Maven asociada al `pom.xml` y ejecuta:

```
mvn -B verify --file bibliotech_v2/pom.xml
```

Maven compila, ejecuta las pruebas y empaqueta el backend. Si una prueba falla, el comando termina con un código de error y GitHub debe marcar el trabajo como fallido. El paso de publicación de reportes usa `if: always()` para conservar `bibliotech_v2/target/surefire-reports/` aun cuando existan pruebas fallidas. El artefacto se llama `reportes-pruebas-backend` y se conserva durante 14 días. Si un error previo impide generar reportes, ese paso emite una advertencia.

El flujo se centra en el backend que implementa la regla. No configura un servicio PostgreSQL externo porque las pruebas usan repositorios simulados o H2 con el perfil `test`.

6. Ramas publicadas y pull request

El remoto `origin` apunta a jorgecahui/BiblioTech. El repositorio comenzó sin historial y se preparó el commit `3421075`, `chore: inicializar BiblioTech`, que contiene solo `.gitignore`. Se crearon `main`, `develop` y `feature/nueva-funcionalidad`, esta última desde `develop`, y se publicaron las tres ramas. La aplicación, la funcionalidad, las pruebas, el flujo CI y los documentos se publicaron en el commit `c449dbd`.

La preparación y publicación se realizaron con la terminal y la API de GitHub. El PDF pide usar **GitHub Desktop**; esa actividad no se realizó y sus capturas quedan pendientes. Los resultados equivalentes de ramas, commits y publicación sí están disponibles en GitHub. No se deben recrear las ramas ni el commit inicial. El nombre `feature/nueva-funcionalidad` respeta el indicado en el PDF.

Se abrió el pull request #1 con **base** `develop` y **compare** `feature/nueva-funcionalidad`. Como las ramas base contienen únicamente `.gitignore`, este primer PR incorpora también la aplicación base. El PR continúa abierto, sin revisión independiente ni integración.

GitHub Actions terminó en **Success** tanto para el push de la implementación como para el pull request. Se conservan la captura del CI del PR y los registros de la ejecución correcta. La revisión debe comprobar la regla de frontera y estos resultados antes de integrar.

El registro de actividades y sus comandos equivalentes están en GUIA_GITHUB.md. La guía distingue los resultados ya publicados de las actividades pendientes de revisión y de las evidencias de GitHub Desktop.

7. Demostración remota de Failure y Success completada

La rama `practica/fallo-limite` se creó desde la versión validada de `feature/nueva-funcionalidad`. Se usó esa base porque el PR hacia `develop` permanece abierto para revisión. El error deliberado se mantiene fuera de la rama funcional y de las ramas base.

1. Se sustituyó únicamente `prestamosActivos >= MAX_PRESTAMOS_ACTIVOS` por `prestamosActivos > MAX_PRESTAMOS_ACTIVOS` dentro de `puedePrestar`. Se mantuvieron el límite en tres y la comprobación de registro.
2. Se publicó el commit de error `34a1f9d`, que activó la ejecución #3. Comenzó el **22/10/2026 a las 15:59:15** y terminó a las **15:59:55** (America/Lima) en **Failure**.
3. El reporte oficial de Surefire registró **27 pruebas, 3 fallos, 0 errores y 0 omitidas**. Fallaron el caso unitario con tres préstamos de `rechazaValidarDesdeElTercerPrestamoActivo` y las pruebas de integración `devolverUnPrestamoLiberaCupo` y `permiteElTerceroYRechazaElCuartoSinDescontarOtroEjemplar`. Las tres aserciones indicaron `expected: <false> but was: <true>`. Se conservaron la captura Failure, los registros y el artefacto oficial de reportes.
4. Se restauró `>=` y se publicó la corrección `ee44be3` en la misma rama. Las expectativas de las pruebas se mantuvieron intactas.
5. La ejecución #4 comenzó a las **16:00:44** y terminó a las **16:01:25** (America/Lima) en **Success**. El reporte oficial descargado verificó **27 pruebas, 0 fallos, 0 errores y 0 omitidas**. Se conservaron la captura Success, los registros y los reportes oficiales. La rama de demostración quedó corregida.

El fallo remoto correspondió a las aserciones de frontera de la regla de negocio, no a un error de descarga, configuración de Java o compilación. La comparación corregida recuperó la ejecución completa. La rama de demostración no necesita integrarse: la versión correcta está publicada y validada en la rama funcional, pendiente de revisión mediante el PR #1.

8. Verificación final local

La verificación completa del backend terminó el **22/10/2026 a las 15:48:32 (America/Lima)** en **BUILD SUCCESS**, con **27 pruebas ejecutadas, 0 fallos y 0 errores**: 18 unitarias, 5 de integración, 3 del contrato HTTP y 1 de carga del contexto. El registro se conserva en backend-success.log.

En una copia aislada del backend se cambió únicamente `>=` por `>` en `puedePrestar`. Se ejecutaron 18 pruebas: **1 falló y 0 tuvieron errores**. La aserción del caso con tres préstamos indicó `expected: <false> but was: <true>`.

Después se restauró >=: **18 pruebas aprobadas, 0 fallos y 0 errores**. La copia restaurada se comparó con el código original mediante SHA-256. El error deliberado no quedó en la aplicación entregada.

La interfaz terminó con **24 pruebas aprobadas en 3 archivos** y compilación de producción correcta, con un paquete inicial de 268.60 kB. Estos resultados y la demostración aislada del fallo corresponden a ejecuciones locales. Las evidencias de GitHub Actions se registran por separado en las secciones 6, 7 y 9.

Elemento	Resultado final
Versión de Java y Maven utilizada	Java 21.0.8 y Apache Maven 3.9.11
Comando final de verificación	`mvn -B verify --file bibliotech_v2/pom.xml`
Resultado de compilación y empaquetado	BUILD SUCCESS, 22/10/2026 15:48:32 Lima
Pruebas ejecutadas / fallos / errores	27 / 0 / 0
Fallo deliberado local	18 pruebas, 1 fallo de aserción, 0 errores; salida 1
Corrección local	18 pruebas, 0 fallos, 0 errores; salida 0
Entorno de la interfaz	Node 22.23.2 y npm 10.9.8
Pruebas y compilación de la interfaz	24/24 aprobadas; compilación correcta
Ubicación de los reportes	`bibliotech_v2/target/surefire-reports/`, después de ejecutar pruebas

Registros de ejecución entregados

Evidencia local	Archivo
Verificación completa del backend	backend-success.log
Detección del error controlado	backend-failure-controlado.log
Verificación después de restaurar la regla	backend-success-corregido.log
Pruebas de Angular	frontend-tests.log
Compilación de Angular	frontend-build.log

Los reportes XML de la demostración están en `docs/evidencias/failure/` y `docs/evidencias/success/`. Para repetir los comandos de npm en este equipo se utilizó `C:/Program Files/nodejs/npm.cmd`, porque el wrapper global de PowerShell apuntaba a una instalación inexistente. La configuración global del equipo no se modificó.

9. Evidencias remotas y actividades pendientes

Evidencia requerida	Estado	Referencia
Ramas main, develop y feature/nueva-funcionalidad en GitHub	Publicadas	Repositorio
Uso y capturas de GitHub Desktop	Pendiente	Las operaciones se realizaron mediante terminal y API de GitHub
Commit de la funcionalidad con sus pruebas	Publicado	c449dbd
Pull request hacia develop	Abierto para revisión	PR #1
CI de la implementación correcta	Success	Push, PR, captura
Error deliberado en practica/fallo-limite	Failure; 27 pruebas, 3 fallos y 0 errores	Commit 34a1f9d, Actions, captura
Corrección de la demostración	Success; 27 pruebas, 0 fallos y 0 errores	Commit ee44be3, Actions, captura
Registros de CI correcto	Conservados	actions-success-inicial.zip
Reportes de Surefire del fallo remoto	Descargados y conservados	reportes-failure.zip, registros
Reportes de Surefire de la corrección remota	Descargados y verificados	reportes-success-corregido.zip, registros
Resumen de conteos JUnit remotos	Conservado	resumen-junit-remoto.json
Revisión independiente e integración del PR	Pendientes	PR #1, sin merge

10. Análisis de la práctica

El caso de dos préstamos comprueba que el sistema permite completar el cupo. El de tres comprueba la frontera donde debe comenzar el rechazo. Si se cambiara `>=` por `>`, los casos con cuatro u ocho seguirían rechazándose, pero el caso exacto de tres revelaría el defecto. Por ello la prueba de frontera es la evidencia principal de la demostración.

Integrar esta comprobación en cada push y pull request permite detectar una modificación incompatible antes de integrar el código. La protección de `develop`, cuando esté configurada para exigir el check y la revisión, convierte el resultado automático en una condición para aceptar los cambios. La transacción y los bloqueos complementan la regla al comprobar el cupo nuevamente durante el registro, cuando ya pueden existir solicitudes concurrentes.

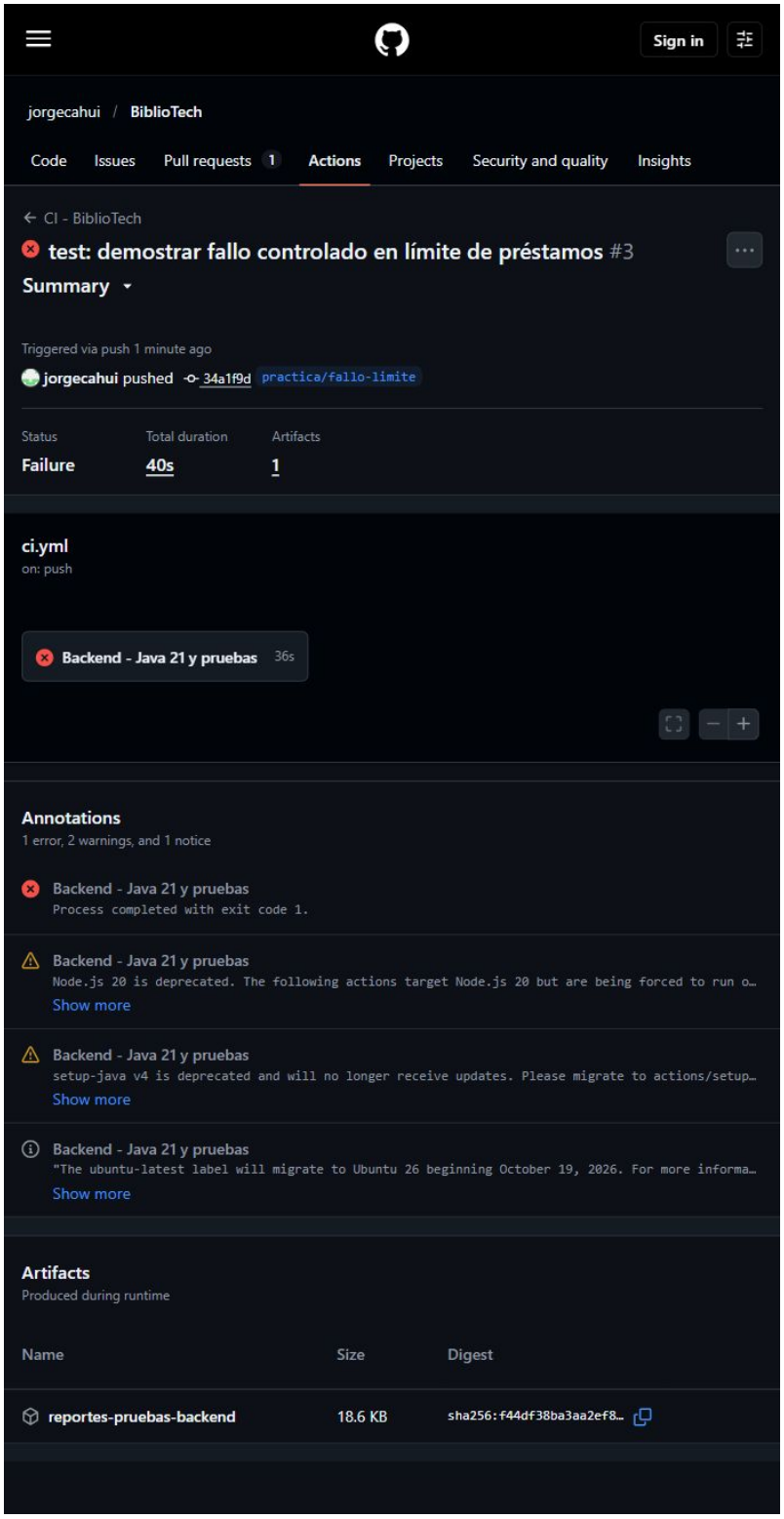
Anexo de evidencias de GitHub Actions

A. Failure: detección del error de frontera

Estado: Failure | **Run:** 39853501576 | **Commit:** `34a1f9d`

27 pruebas; 3 fallos; 0 errores. Cambio deliberado de >= a >.

Abrir ejecución real en GitHub Actions



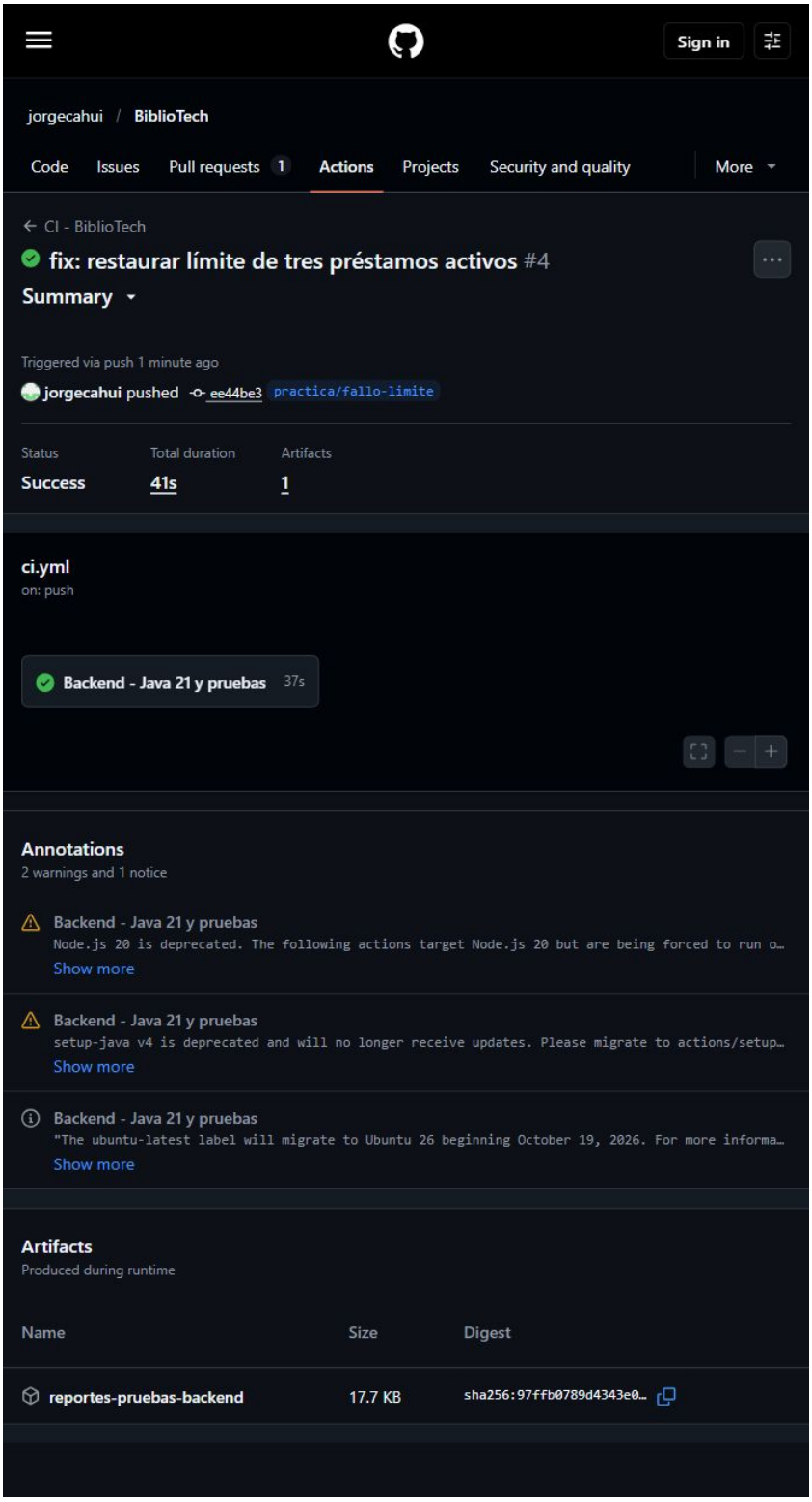
Anexo de evidencias de GitHub Actions

B. Success: regla restaurada

Estado: Success | **Run:** 39853717419 | **Commit:** `ee44be3`

27 pruebas; 0 fallos; 0 errores. Comparación >= restaurada.

Abrir ejecución real en GitHub Actions



Anexo de evidencias de GitHub Actions

C. Success: integración continua del PR #1

Estado: Success | **Run:** 39853394870 | **Commit:** `c449dbd`

CI del pull request hacia develop. Funcionalidad, pruebas y compilación correctas.

Abrir ejecución real en GitHub Actions

